



Mapeamento Objeto-Relacional Moderno e Tipado

Integração Eficiente entre APIs JavaScript e Bancos de Dados Relacionais

IFSUL - Campus Passo Fundo | Ciência da Computação (PW)

Professora: Jéssica Di Domênico

Integrantes: Daniel Silva e Guilherme Vignochi

O Conceito de ORM (Object-Relational Mapping)

- **Problema:** O desalinhamento entre o modelo orientado a objetos do JavaScript e as tabelas relacionais do SQL.
- **Definição:** Camada de abstração que permite consultar e persistir dados usando objetos da linguagem.
- **Menos Boilerplate:** Elimina queries repetitivas, tratamento bruto de conexões e conversões manuais.
- **Segurança por Padrão:** Prevenção nativa contra injeção de SQL (SQL Injection) via parametrização automática.
- **Modelagem Code-First:** O esquema e os tipos do banco vivem diretamente no repositório de código.

Por que o Drizzle ORM?

- **Filosofia Central:** 'If you know SQL, you know Drizzle' — zero abstrações misteriosas ou lentas.
- **Performance Extrema:** Quase sem overhead de execução, gerando SQL praticamente idêntico ao nativo.
- **TypeScript & JS First:** Inferência automática de tipos a partir dos schemas sem dependência de geradores pesados.
- **Flexibilidade Total:** Oferece tanto o SQL-like Query Builder quanto a Relational Queries API.
- **Serverless Ready:** Sem inicialização demorada (zero cold-start), ideal para microsserviços modernos.

Drizzle vs Outros ORMs Tradicionais

Critério	Drizzle ORM	Prisma	TypeORM
Arquitetura	Leve / Modular	Binário em Rust (pesado)	Abstração clássica / Decorators
Performance	Próxima ao SQL puro	Média (overhead de bridge)	Moderada
Sintaxe	Muito próxima ao SQL	Sintaxe própria (.prisma)	Active Record / Data Mapper
Cold Start	Instantâneo (0ms)	Lento em serverless	Médio

Configuração do Ambiente e Instalação

- **Banco de dados utilizado:** PostgreSQL.
- **drizzle-orm:** Módulo central para construção de queries e integração de drivers.
- **pg (node-postgres):** Driver de conexão nativo do ecossistema Node.js.
- **drizzle-kit:** Ferramenta CLI de migrações e painel visual de administração.
- Comando de instalação:
 - `npm install drizzle-orm pg`
 - `npm install -D drizzle-kit`

Criação do Banco de Dados

```
banco-animais.sql

-- Criação do Banco de Dados
CREATE DATABASE "banco-animais";

-- Criação da Tabela animais
CREATE TABLE IF NOT EXISTS animais (
    id SERIAL PRIMARY KEY,
    nome VARCHAR(100) NOT NULL,
    especie VARCHAR(50) NOT NULL,
    idade INTEGER NOT NULL
);
```

Inicialização do Projeto

Terminal

```
1 # Criar a pasta do projeto e entra nela
2 mkdir api-animais-drizzle
3 cd api-animais-drizzle
4
5 # Inicializa o arquivo package.json
6 npm init -y
```

No package.json adicionar

package.json

```
{  
  "name": "api-animais-drizzle",  
  "type": "module",  
  "main": "index.js",  
  "scripts": {  
    "start": "node index.js"  
  }  
}
```

Instalar dependências

Terminal

```
1 # Express para o servidor HTTP, Drizzle a biblioteca ORM, pg driver
  do PostgreSQL para Node.js
2 npm install express drizzle-orm pg
3
4 # Ferramenta opcional, de linha de comando para migrações
  automáticas ou para abrir o painel visual no caso o Drizzle Studio
5 npm install -D drizzle-kit
```

Definição de Esquema (schema.js)

```
schema.js

1 import { pgTable, serial, varchar, integer } from 'drizzle-orm/pg-
  core';
2
3 export const animais = pgTable('animais', {
4   id: serial('id').primaryKey(),
5   nome: varchar('nome', { length: 100 }).notNull(),
6   especie: varchar('especie', { length: 50 }).notNull(),
7   idade: integer('idade').notNull()
8 });
```

Acesso à Interface Web do Drizzle (drizzle.config.js)

```
drizzle.config.js

import { defineConfig } from "drizzle-kit";

export default defineConfig({
  schema: "./schema.js",      // Caminho para o arquivo onde está o pgTable
  dialect: "postgresql",      // Tipo do banco
  dbCredentials: {
    url: "postgres://postgres:sua_senha@localhost:5432/banco-animais",
  },
});
```

Importações necessárias (index.js)

```
index.js

1 import express from 'express';
2 import { drizzle } from 'drizzle-orm/node-postgres';
3 import pg from 'pg';
4 import { animais } from './schema.js';
5 import { eq } from 'drizzle-orm';
6
7 const app = express();
8 app.use(express.json());
9
10 ...
```

Conexão com o PostgreSQL (index.js)

index.js

```
const pool = new pg.Pool({
  connectionString: 'postgres://postgres:senha@localhost:5432/banco-animais'
});
const db = drizzle(pool);

...
```

Rota para Listar todos os animais (index.js) (SELECT * FROM animais)

```
index.js

1 app.get('/animais', async (req, res) => {
2   const lista = await db.select().from(animais);
3   res.json(lista);
4 });
5
6 ...
```

Rota para Cadastrar animal (index.js) (INSERT INTO animais VALUES (...))

```
index.js

1 app.post('/animais', async (req, res) => {
2   const { nome, especie, idade } = req.body;
3   await db.insert(animais).values({ nome, especie, idade });
4   res.status(201).json({ mensagem: 'Animal cadastrado com
  sucesso!' });
5 });
6
7 ...
```

Atualização / Edição (index.js)

```
index.js
1 // PUT /animais/:id - atualizar dados de um animal existente
2 app.put('/animais/:id', async (req, res) => {
3   try {
4     const id = parseInt(req.params.id);
5     const { nome, especie, idade } = req.body;
6
7     // Executa o UPDATE no PostgreSQL filtrando pelo id
8     const resultado = await db
9       .update(animais)
10      .set({
11        nome: nome,
12        especie: especie,
13        idade: Number(idade)
14      })
15      .where(eq(animais.id, id))
16      .returning(); // Retorna o registro alterado
17
18     // Se o array voltar vazio, significa que o ID não existia no banco
19     if (resultado.length === 0) {
20       return res.status(404).json({ mensagem: 'Animal não encontrado para atualização.' });
21     }
22
23     res.status(200).json({
24       mensagem: 'Animal atualizado com sucesso!',
25       animal: resultado[0]
26     });
27   } catch (erro) {
28     res.status(500).json({ erro: 'Erro ao atualizar o animal.' });
29   }
30 });
31
32 ...
```

Exclusão / Remoção (index.js)

```
index.js
1 // DELETE /animais/:id - Deletar um animal pelo ID
2 app.delete('/animais/:id', async (req, res) => {
3   try {
4     const id = parseInt(req.params.id);
5
6     // Executa o DELETE no PostgreSQL filtrando pelo id
7     const resultado = await db
8       .delete(animais)
9       .where(eq(animais.id, id))
10      .returning(); // Confirma qual registro foi excluído
11
12     if (resultado.length === 0) {
13       return res.status(404).json({ mensagem: 'Animal não encontrado para
remoção.' });
14     }
15
16     res.status(200).json({ mensagem: 'Animal removido com sucesso!' });
17   } catch (erro) {
18     res.status(500).json({ erro: 'Erro ao remover o animal.' });
19   }
20 });
21
22 ...
```

Configuração da Porta e Inicialização da API (index.js)

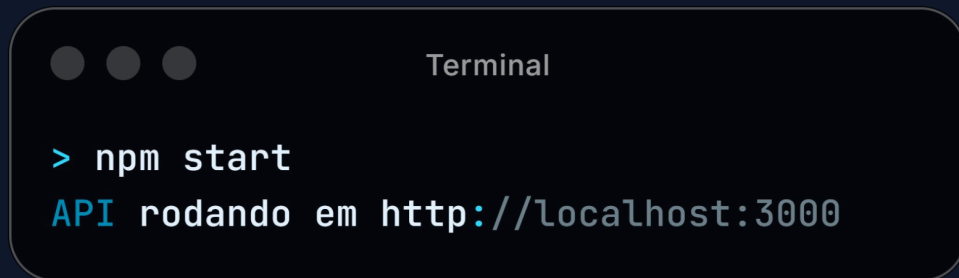


index.js

```
1 app.listen(3000, () => console.log('API rodando em  
http://localhost:3000'));
```

Execução da API Express

No terminal, dentro da pasta do projeto, execute: `npm start`



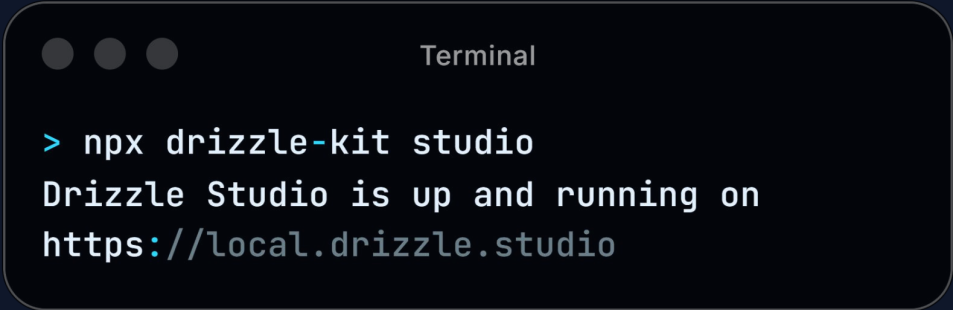
```
Terminal  
  
> npm start  
API rodando em http://localhost:3000
```

O console exibirá:

API rodando em `http://localhost:3000`

Execução do Drizzle Studio

No terminal, dentro da pasta do projeto, execute: `npx drizzle-kit studio`



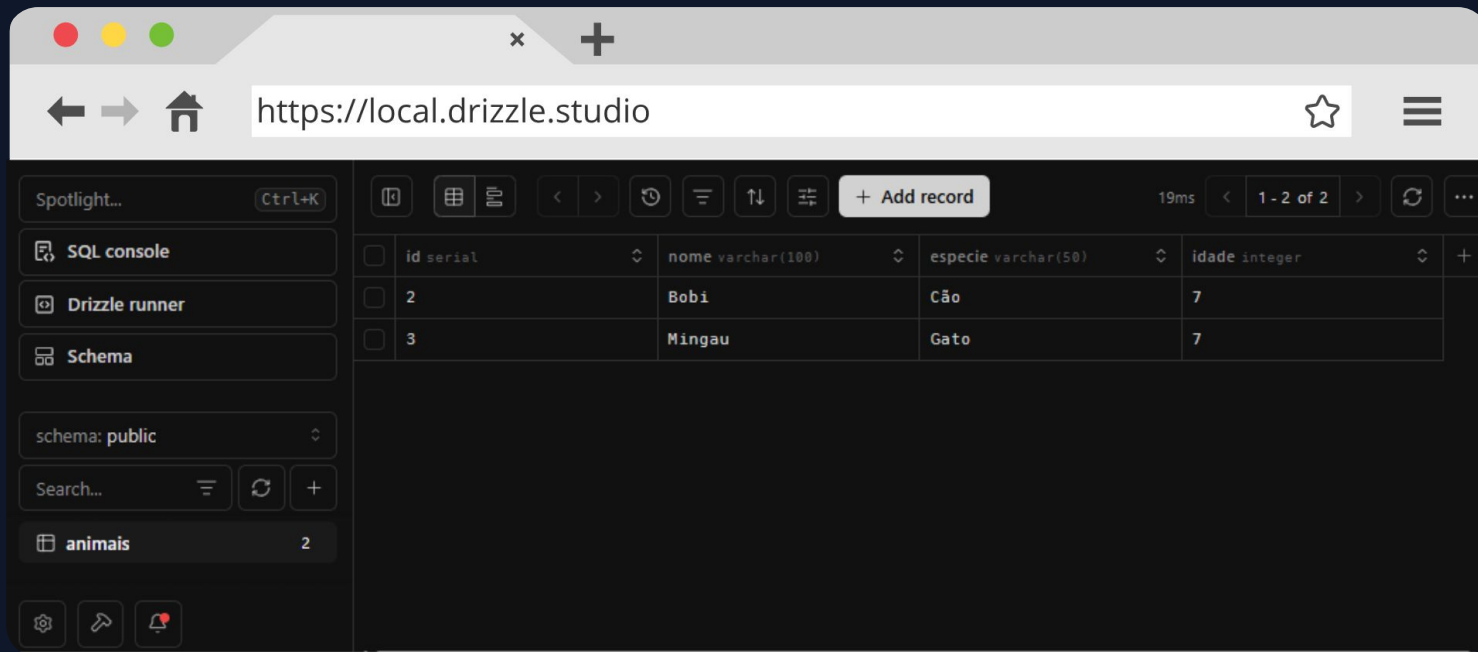
```
Terminal  
  
> npx drizzle-kit studio  
Drizzle Studio is up and running on  
https://local.drizzle.studio
```

O console exibirá:

Drizzle Studio is up and running on **`https://local.drizzle.studio`**

Acesse o endereço para abrir a interface web de gerenciamento do banco (estilo pgAdmin).

Drizzle Studio em execução



Produtividade: Drizzle Kit & Studio

- **Drizzle Kit:** Automatiza a geração de migrações SQL com base no código do schema.js.
- **Comando para gerar migração:** `npx drizzle-kit generate`
- **Drizzle Studio:** Interface visual de auditoria do banco no navegador:
 - `npx drizzle-kit studio`
- Dispensa a necessidade de softwares clientes pesados como pgAdmin/DBeaver durante o desenvolvimento.

Considerações Finais

- **Curva de Aprendizado:** Mínima para quem já compreende os fundamentos de SQL.
- **Controle e Transparência:** Sem queries ocultas em segundo plano ou surpresas de desempenho.
- **Segurança:** Tipagem de ponta a ponta e consultas parametrizadas por padrão.
- **Veredito:** Excelente alternativa para substituir o SQL bruto em APIs Express corporativas.

Dúvidas

Documentação oficial: [Drizzle ORM](#)

Repositório do Trabalho: [Github - Trabalho](#)

Repositório Github

